PROJECT 1 REPORT

Ticket Booking System using TDD

---

1. Test-Driven Development (TDD)

In this project, we followed the TDD process discussed in the course slides.

The development process followed the cycle:

Red → Green → Refactor

First, we wrote unit tests for each user story before implementing the code. The tests initially failed **(**Red**).** Then we implemented the minimum code needed to make the tests pass **(**Green**).** Finally, we improved and cleaned the code structure (Refactor). This approach helped us focus only on required functionality and avoid unnecessary code.

In our implementation:

- `happyPath_validBooking_returnsTrue()`
- `invalidInput_paymentBookingNotificationNeverCalled()`
- `soldOut_eventChecked_paymentBookingNotificationNeverCalled()`

were implemented before finalizing the production code.

---

2. Dependency Injection

According to the Test Doubles slides, Dependency Injection allows dependencies to be passed through constructors or setter methods instead of being created inside the class. This improves software testability.

In our code, `BookingManager` receives:

- `IPaymentGateway`
- `INotificationService`
- `IEventRepository`

through the constructor:

```
BookingManager(paymentGateway, notificationService, eventRepository)
```

This allowed us to replace real dependencies with mock objects during testing.

---

3. Mocking and Test Doubles

The course slides explain that test doubles are used when real dependencies are unavailable, expensive, nondeterministic, or may cause unwanted side effects.

In our project, we used Mockito to create mocks for:

- Payment Gateway
- Notification Service
- Event Repository

Example behaviors tested:

- `times(1)` → verify method called exactly once
- `never()` → verify method was not called

This allowed us to isolate `BookingManager` and test only its logic.

---

4. Software Testability

The test automation slides explain that testability depends on:

- **Controllability** → ability to control test inputs
- **Observability** → ability to observe system outputs and behavior

Our design improved controllability by injecting mocks, and improved observability by verifying interactions with dependencies.

---

5. Team Contribution Matrix

Team Member	Contribution
Farah Abdullah younis 2335869	Implemented `BookingManager.java`,wrote JUnit tests, applied TDD, implemented mocks and project logic
Areen jamil farah 2141323	Implemented `BookingManagerTest.java`, JUnit tests, Mockito mocks